**TRƯỜNG ĐẠI HỌC BÁCH KHOA**

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**

# BÀI TẬP LỚN THIẾT KẾ HỆ THỐNG NHÚNG

**ĐỀ TÀI:**

**Gesture-Controlled CAR**

**LỚP L04 --- NHÓM 13 --- HK 251**

**Giảng viên hướng dẫn: Nguyễn Phan Hải Phú**

| **Sinh viên thực hiện** | **Mã số sinh viên** | **Đóng góp** |
|---|---|---|
| Lâm Trọng Nhân | 2312423 | 100% |
| Đàm Lê Khải | 2311545 | 100% |
| Trần Doãn Cường | 2310404 | 100% |
| Nguyễn Nhật Minh | 2312092 | 100% |

*Thành phố Hồ Chí Minh – 2025*

**Chương 1: Giới thiệu** ..... 3

***1.1 Tổng quan*** ..... ***3***

***1.2 Nhiệm vụ đề tài*** ..... ***3***

**2. Lý thuyết** ..... 4

***2.1 Tổng quan về Bluetooth communication*** ..... ***4***

***2.2 Cảm biến MPU6050*** ..... ***5***

***2.3 Bộ điều khiển động cơ*** ..... ***5***

***2.4 Cơ chế điều khiển xe qua Bluetooth*** ..... ***5***

**3. Thiết kế và thực hiện phần cứng** ..... 5

***3.1 Yêu cầu thiết kế*** ..... ***5***

***3.2 Phân tích*** ..... ***6***

***3.4 Sơ đồ khối chi tiết và giải thích*** ..... ***7***

***3.5 Tính toán và sơ đồ mạch chi tiết*** ..... ***8***

**Chương 4: Thiết kế và thực hiện phần mềm** ..... 8

***4.1 Yêu cầu phần mềm*** ..... ***8***

***4.2 Lưu đồ giải thuật găng tay*** ..... ***9***

***4.3 Lưu đồ giải thuật xe*** ..... ***14***

**Chương 5: Kết quả thực hiện** ..... 20

***5.1 Tiến hành thử nghiệm:*** ..... ***20***

***5.2 Kết quả thí nghiệm*** ..... ***21***

***5.3 Phân tích kết quả thử nghiệm*** ..... ***21***

**Chương 6: Kết luận và hướng phát triển** ..... 22

***6.1 Kết luận*** ..... ***22***

***6.2 Hướng phát triển*** ..... ***22***

**Tài liệu tham khảo** ..... 23

**Phụ lục** ..... 23

# Chương 1: Giới thiệu

## *1.1 Tổng quan*

Trong những năm gần đây, các hệ thống điều khiển không dây đã trở nên phổ biến nhờ tính linh hoạt, phạm vi ứng dụng rộng và dễ triển khai. Một trong những công nghệ truyền thông không dây đơn giản, tiết kiệm năng lượng và được sử dụng rộng rãi nhất là Bluetooth. Với ưu điểm dễ tích hợp vào các hệ thống nhúng, Bluetooth cho phép truyền dữ liệu nhanh giữa thiết bị điều khiển và thiết bị được điều khiển.

Đề tài "Bluetooth communication" ứng dụng để làm project "Xe điều khiển bằng cử chỉ thông qua giao tiếp Bluetooth" hướng đến việc xây dựng một mô hình xe thông minh có khả năng nhận lệnh điều khiển dựa trên cử chỉ tay người dùng. Lệnh cử chỉ được cảm biến ghi nhận, xử lý và truyền qua Bluetooth đến vi điều khiển trên xe, từ đó tạo ra chuyển động tương ứng.

Mục tiêu nghiên cứu tập trung vào:

- Tìm hiểu cơ chế giao tiếp Bluetooth giữa thiết bị điều khiển (ví dụ: smartphone, găng tay cảm biến, module MPU6050,…) và xe sử dụng vi điều khiển ESP32.
- Xây dựng bộ điều khiển cử chỉ đơn giản dựa trên cảm biến gia tốc/gyro.
- Thiết kế hệ thống điều khiển xe đáp ứng chính xác các thao tác thực tế.
- Hiện thực mô hình hoàn chỉnh có thể hoạt động ổn định và trực quan.

Từ đó, đề tài đặt ra nhiệm vụ nghiên cứu, thiết kế và thử nghiệm hệ thống giao tiếp Bluetooth kết hợp thuật toán nhận dạng cử chỉ để điều khiển xe hai hoặc bốn bánh.

## *1.2 Nhiệm vụ đề tài*

Để hoàn thành mục tiêu, các nhiệm vụ chính cần thực hiện bao gồm:

Nội dung 1: Tìm hiểu lý thuyết về Bluetooth Communication

- Tìm hiểu nguyên lý hoạt động Bluetooth Classic/ BLE.
- Cách thiết lập liên kết (pairing), truyền dữ liệu dạng chuỗi.
- Nghiên cứu hoạt động của module Bluetooth (HC-05/HC-06 hoặc BLE của ESP32).
- Nắm được tốc độ truyền, giới hạn khoảng cách, cấu trúc frame dữ liệu.
- Ý tưởng tiếp cận: Xây dựng giao tiếp một chiều hoặc hai chiều giữa bộ điều khiển và xe, dùng chuẩn UART để giao tiếp giữa module Bluetooth và vi điều khiển.

Nội dung 2: Tìm hiểu về Kit điều khiển và phần cứng cảm biến

- Khảo sát các nền tảng phù hợp: Arduino UNO/Nano, ESP32, STM32.
- Tìm hiểu cảm biến cử chỉ hoặc IMU: MPU6050/MPU9250.
- Tìm hiểu driver motor: L298N, L293D.

- Xác định yêu cầu nguồn, tốc độ xử lý và khả năng giao tiếp.

Ý tưởng thực hiện: Dùng MPU6050 gắn trên găng tay để thu dữ liệu nghiêng/ gia tốc → gửi tới vi điều khiển → chuyển lệnh qua Bluetooth đến xe.

Nội dung 3: Thiết kế bộ điều khiển cử chỉ sử dụng thuật toán xử lý tín hiệu

- Thu thập dữ liệu cảm biến (gia tốc, vận tốc góc).
- Tính góc nghiêng bằng thuật toán complementary filter hoặc Kalman.
- Xác định ngưỡng và quy tắc cử chỉ:
- Nghiêng tay về trước → xe tiến
- Nghiêng tay về sau → xe lùi
- Nghiêng trái/phải → xe rẽ
- Tư thế trung lập → xe dừng

Ý tưởng tiếp cận: Đọc IMU → Lọc nhiễu → Tính góc → Mã hóa lệnh → Gửi Bluetooth → Xe nhận → Điều khiển động cơ.

Nội dung 4: Thiết kế và xây dựng mô hình xe điều khiển

- Lắp ráp khung xe, driver motor và module Bluetooth.
- Lập trình Arduino/ESP32 nhận dữ liệu Bluetooth và điều khiển motor.
- Thiết kế bộ khung hợp lý, tránh nhiễu từ trường ảnh hưởng IMU.

## 2. Lý thuyết

### *2.1 Tổng quan về Bluetooth communication*

Bluetooth là một công nghệ truyền thông không dây tầm ngắn (short-range wireless communication) sử dụng sóng vô tuyến ở băng tần 2.4 GHz ISM để kết nối các thiết bị điện tử như điện thoại, máy tính, cảm biến, thiết bị IoT,… Công nghệ này cho phép truyền dữ liệu với mức tiêu thụ năng lượng thấp, chi phí rẻ, và không cần đường truyền hữu tuyến.

Bluetooth hoạt động dựa trên mô hình master–slave (central–peripheral), trong đó một thiết bị quản lý kết nối và các thiết bị còn lại thực hiện truyền nhận dữ liệu. Tùy theo phiên bản, Bluetooth hỗ trợ nhiều tốc độ và phạm vi khác nhau, nổi bật với hai dòng chính:

- Bluetooth Classic (BR/EDR): phù hợp truyền dữ liệu liên tục như âm thanh.
- Bluetooth Low Energy (BLE): tối ưu cho thiết bị tiêu thụ điện thấp như cảm biến, wearable, robot điều khiển từ xa.

Quá trình giao tiếp Bluetooth bao gồm quét thiết bị, ghép nối (pairing), thiết lập kết nối và trao đổi dữ liệu. Nhờ thiết kế đơn giản và khả năng tương thích cao, Bluetooth

trở thành nền tảng phổ biến trong các ứng dụng di động, điều khiển, tự động hóa, IoT và các hệ thống nhúng.

### *2.2 Cảm biến MPU6050*

MPU6050 là cảm biến gồm:

- 3 trục gia tốc
- 3 trục gyro
- Giao tiếp I2C

Hai thuật toán thường dùng:

- Complementary Filter (nhanh, nhẹ)
- Kalman Filter (chính xác cao hơn)

Góc nghiêng được tính từ:

- Gia tốc kế → góc tương đối theo trọng lực
- Con quay hồi chuyển → tốc độ quay tích phân theo thời gian

Kết hợp hai tín hiệu cho kết quả ổn định.

### *2.3 Bộ điều khiển động cơ*

Driver L298N hoạt động theo nguyên lý cầu H, cho phép:

- Điều khiển 2 động cơ DC độc lập.
- Thay đổi chiều quay và tốc độ thông qua PWM.

### *2.4 Cơ chế điều khiển xe qua Bluetooth*

Quy trình truyền – nhận dữ liệu:

1. Bộ điều khiển cử chỉ → xử lý → tạo mã lệnh (ví dụ: ‘F’, ‘B’, ‘L’, ‘R’).
2. Gửi lệnh qua Bluetooth.
3. Xe nhận chuỗi ký tự.
4. Arduino/ESP32 phân tích và kích hoạt động cơ tương ứng.

## 3. Thiết kế và thực hiện phần cứng

### *3.1 Yêu cầu thiết kế*

a) Chức năng chính

Nhận cử chỉ (góc nghiêng tay) → mã hóa thành lệnh → truyền không dây qua Bluetooth → xe nhận lệnh → điều khiển động cơ (tiến/lùi/rẽ/dừng).

b) Hiệu năng & định lượng

Tốc độ phản hồi ≤ 200 ms (từ cử chỉ đến di chuyển).

Độ chính xác nhận dạng góc: ±5° sau lọc.

Khoảng cách điều khiển: ≥ 8 m (Bluetooth Classic / BLE).
Thời gian hoạt động pin: ≥ 60 phút (chế độ vận hành bình thường).
Tải động cơ: xe 2 bánh (2 motor DC), vận tốc tối đa mục tiêu ~0.8–1.5 m/s.
c) Yêu cầu phần cứng
MCU: ESP32 — xử lý IMU + Bluetooth.
IMU: MPU6050 (3-axis accel + 3-axis gyro).
Bluetooth: BLE (ESP32 tích hợp) hoặc HC-05 (Classic) nếu dùng Arduino.
Driver motor: L298N (tối thiểu 2A liên tục trên kênh; lưu ý dòng stall).
Nguồn: Pin Li-ion 2S (7.4 V nominal) hoặc pin LiPo 2S; dung lượng ≥ 2000 mAh.
d) An toàn & cơ khí
Bảo vệ ngắn mạch & chống đảo cực.
Kết cấu khung nhỏ gọn, cách ly rung cho IMU.

### *3.2 Phân tích*

**a) Lựa chọn MCU và Bluetooth**

***ESP32***

Ưu điểm: tích hợp Bluetooth (BLE + Classic), CPU mạnh, ADC, PWM, giao tiếp I2C dễ, hoạt động ở 3.3V (không cần level shifter với MPU6050). Hỗ trợ Wi-Fi nếu mở rộng.

Nhược điểm: phức tạp hơn với người mới; tiêu thụ khi dùng Wi-Fi cao (không cần bật Wi-Fi).

Quyết định: chọn ESP32 vì xử lý thuật toán (complementary/Kalman) tốt hơn, BLE/BT tích hợp, đơn giản hóa phần nối IMU.

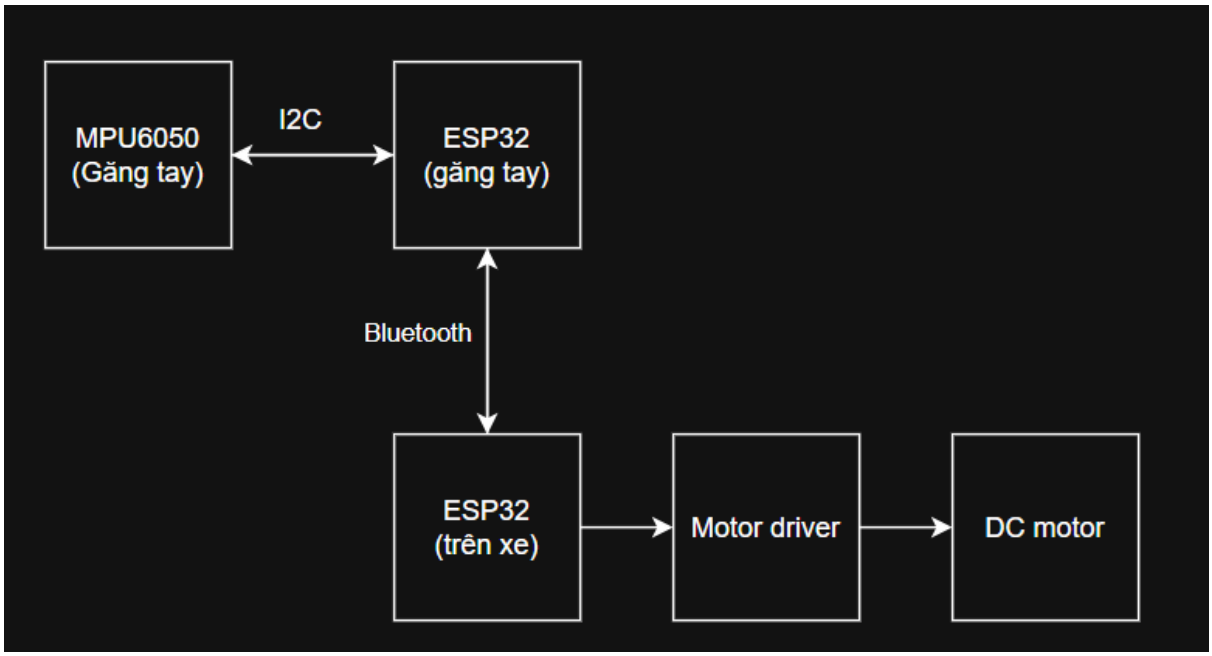
**b) Lựa chọn driver động cơ**

**L298N**

Ưu điểm: rẻ, phổ biến, hỗ trợ đảo chiều bằng cầu H.

Nhược điểm: tổn hao điện lớn (rơi áp ~2–4 V ở dòng cao), nóng, kém hiệu suất cho dòng lớn.

3.3 Sơ đồ khối phần cứng




**Giải thích nhiệm vụ từng khối**

IMU (MPU6050): lấy dữ liệu gia tốc và gyro (AX, AY, AZ, GX, GY, GZ).

MCU trên găng tay: chạy thuật toán lọc (complementary), nhận dạng cử chỉ (ngưỡng), mã hóa lệnh, gửi qua BLE.

Kênh Bluetooth: truyền lệnh dạng đơn giản.

MCU trên xe: nhận lệnh, giải mã, điều khiển motor driver bằng PWM/IN.

Motor driver: chuyển tín hiệu logic thành dòng cho motor, điều khiển chiều & tốc độ.

### *3.4 Sơ đồ khối chi tiết và giải thích*

1) Găng tay (Transmitter)

MPU6050 ↔ ESP32 (I2C: SDA, SCL)

ESP32 xử lý, gửi BLE

2) Xe (Receiver)

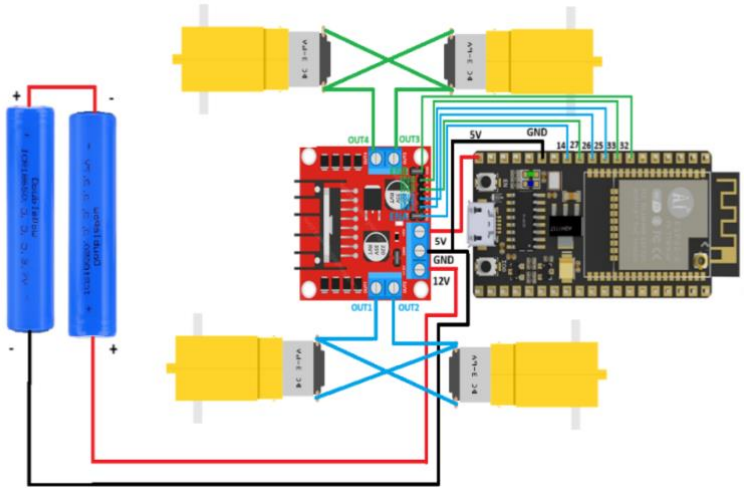
ESP32 (hoặc Arduino + HC-05) làm receiver.

ESP32 UART/Bluetooth ↔ Motor Driver IN1..IN4 (PWM).

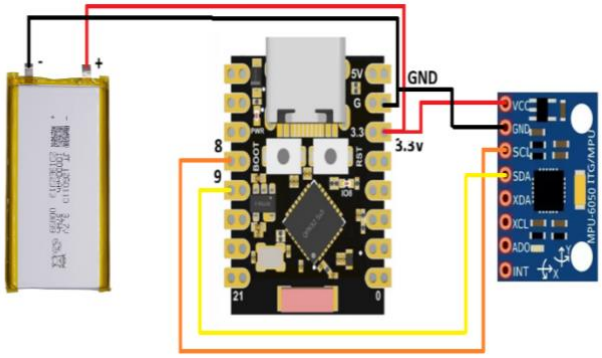
Driver motor (L298N) ↔ 4 motor DC.

### *3.5 Tính toán và sơ đồ mạch chi tiết*

Sơ đồ mạch cho xe:




Sơ đồ mạch cho găng tay:




## Chương 4: Thiết kế và thực hiện phần mềm

### *4.1 Yêu cầu phần mềm*

Phần mềm phải thực hiện các chức năng cốt lõi để đảm bảo hệ thống hoạt động:

FR1. Thu thập dữ liệu cử chỉ (Glove):

FR1.1. Đọc và thu thập dữ liệu từ cảm biến IMU MPU6050 (gia tốc/gyroscope) trên găng tay.

FR1.2. (Dự kiến) Đọc và thu thập dữ liệu từ cảm biến uốn (flex sensor) trên găng tay (nếu được tích hợp).

FR2. Truyền dữ liệu không dây (Glove sang Car):

FR2.1. Thiết bị găng tay (ESP32) phải truyền dữ liệu cảm biến đến thiết bị xe (ESP32) sử dụng giao thức Bluetooth/WiFi.

FR2.2. Đảm bảo kết nối không dây ổn định và liên tục trong quá trình vận hành.

FR3. Giải mã và Điều khiển (Car):

FR3.1. Tiếp nhận dữ liệu từ găng tay.

FR3.2. Giải mã cử chỉ từ dữ liệu cảm biến (ví dụ: góc nghiêng của tay, mức độ uốn của ngón tay) để chuyển thành các lệnh điều khiển.

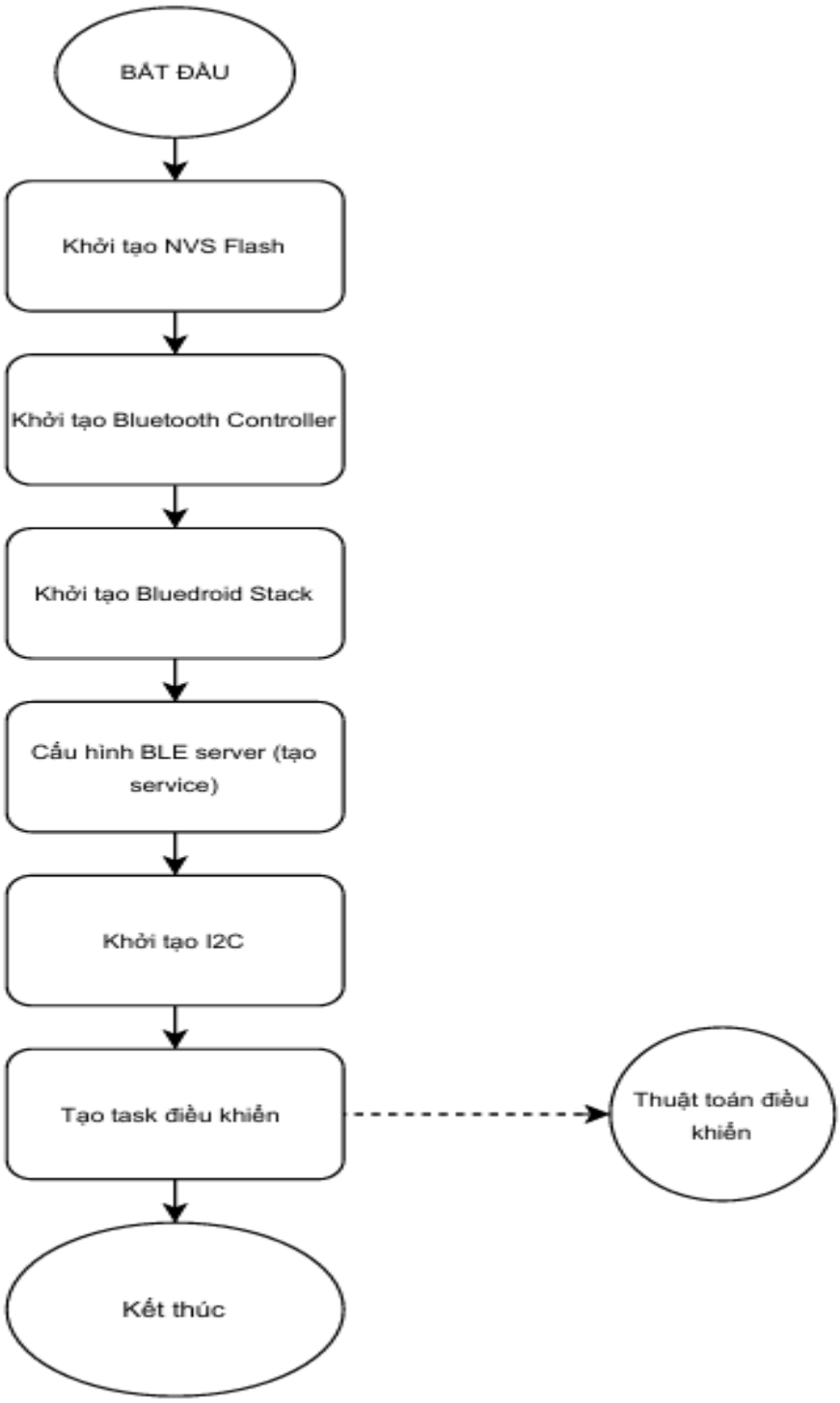
Các lệnh điều khiển tối thiểu: Tiến, Lùi, Rẽ Trái, Rẽ Phải, Dừng.

FR3.3. Điều khiển 4 động cơ DC thông qua Driver TB6612FNG.

Thiết lập PWM để điều khiển tốc độ của động cơ.

### *4.2 Lưu đồ giải thuật găng tay*

**a) Lưu đồ giải thuật tổng quát**




Bắt Đầu:

Đây là điểm khởi đầu của chương trình, thường là hàm main() hoặc app_main() trong môi trường ESP-IDF.

Khởi tạo NVS Flash:

NVS (Non-Volatile Storage) Flash là một bộ nhớ không thay đổi trên ESP32 được sử dụng để lưu trữ dữ liệu vĩnh viễn (ví dụ: thông tin cấu hình, dữ liệu hiệu chuẩn, khóa

Bluetooth). Việc khởi tạo NVS Flash là cần thiết trước khi khởi tạo Bluetooth hoặc WiFi để đảm bảo hệ thống có thể truy cập và lưu trữ các thông số cần thiết.

Khởi tạo Bluetooth Controller:

Bước này khởi động tầng điều khiển vật lý (Controller layer) của Bluetooth, quản lý việc giao tiếp không dây ở cấp độ thấp nhất.

Khởi tạo Bluedroid Stack:

Bluedroid Stack là ngăn xếp phần mềm chịu trách nhiệm thực hiện giao thức Bluetooth Classic và Bluetooth Low Energy (BLE). Việc khởi tạo ngăn xếp này cho phép ứng dụng sử dụng các chức năng giao tiếp Bluetooth ở cấp độ cao hơn.

Cấu hình BLE Server (tạo service):

Trong giao tiếp Bluetooth Low Energy (BLE), thiết bị Găng tay đóng vai trò là Server. Bước này bao gồm:

Tạo Service (Dịch vụ): Định nghĩa các dịch vụ (Service) chứa các đặc tính (Characteristic) để truyền dữ liệu.

Định nghĩa Characteristic: Các đặc tính này đại diện cho dữ liệu cần gửi (ví dụ: dữ liệu góc quay/gia tốc X, Y, Z từ MPU6050, lệnh điều khiển).

Bắt đầu Server: Kích hoạt Server để Client (thiết bị Xe) có thể kết nối.

Khởi tạo I2C:

I2C (Inter-Integrated Circuit) là giao thức giao tiếp nối tiếp được sử dụng để kết nối vi điều khiển ESP32 với các thiết bị ngoại vi, cụ thể ở đây là Cảm biến IMU MPU6050. Bước này thiết lập các chân GPIO (General Purpose Input/Output) làm chân SCL (Clock) và SDA (Data) và cấu hình tốc độ truyền.

Tạo task điều khiển:

Đây là bước quan trọng trong hệ điều hành thời gian thực (RTOS) của ESP32 (FreeRTOS). Một Task (nhiệm vụ) được tạo ra để thực hiện vòng lặp chính của ứng dụng:

Đọc Cảm biến: Đọc dữ liệu từ MPU6050 qua I2C.

Giải mã Cử chỉ: Chuyển đổi dữ liệu cảm biến thành lệnh điều khiển (Tiến, Lùi, Rẽ...) theo Thuật toán điều khiển (tham chiếu bên cạnh).

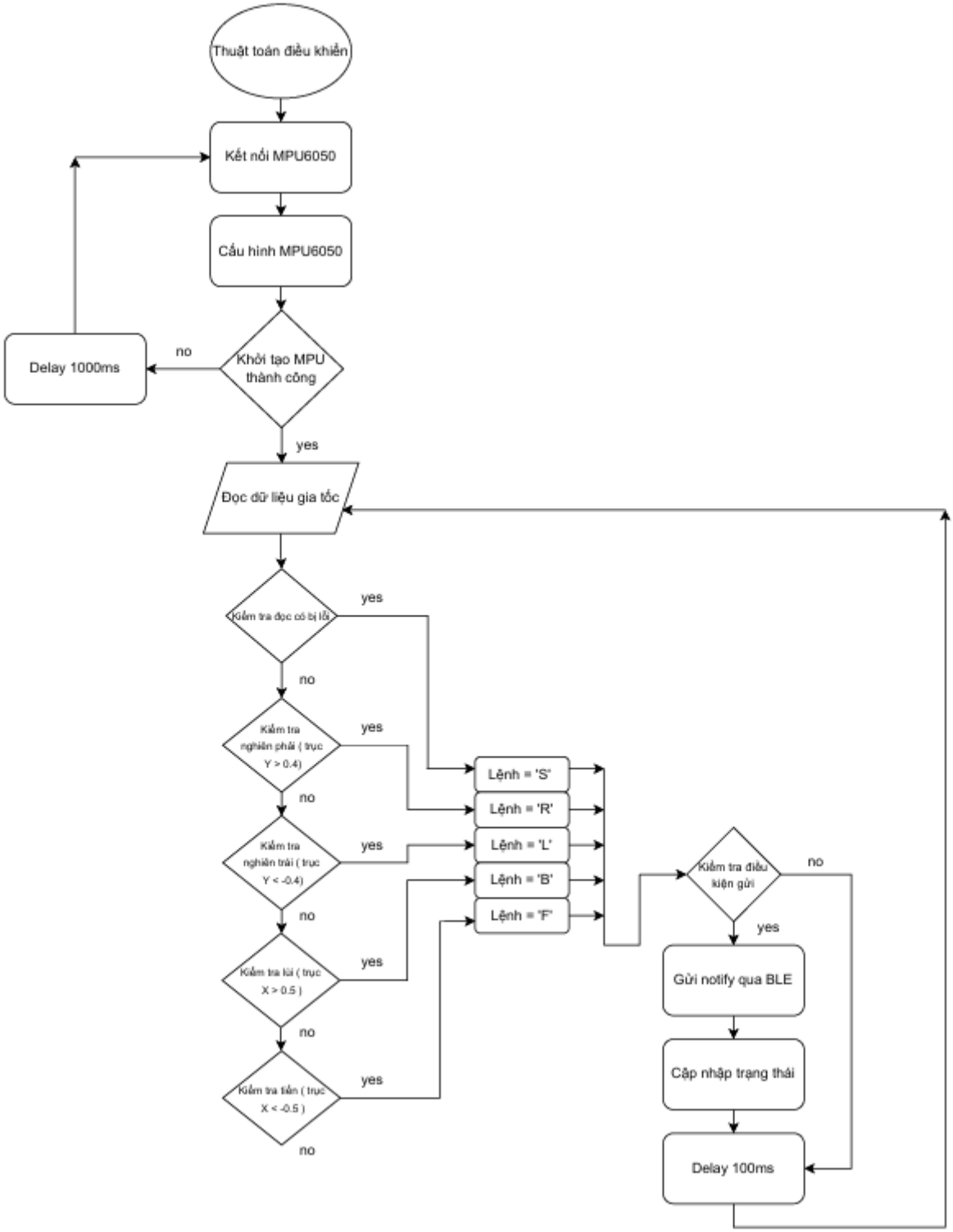
Truyền Lệnh: Gửi lệnh điều khiển đã giải mã đến thiết bị Xe thông qua BLE Characteristic đã cấu hình.

Mũi tên đứt nét từ "Tạo task điều khiển" trỏ đến Thuật toán điều khiển cho thấy Task này sẽ thực thi logic của thuật toán đó.

Kết thúc:

Điểm cuối của quá trình khởi tạo. Sau bước này, hệ thống sẽ chạy liên tục vòng lặp chính trong Task điều khiển đã tạo.

**b) Lưu đồ giải thuật chi tiết**




Thuật toán điều khiển:

Điểm bắt đầu của logic điều khiển chính sau khi hệ thống đã được khởi tạo.

Kết nối MPU6050:

Hệ thống cố gắng thiết lập giao tiếp với cảm biến MPU6050 (cảm biến 6 bậc tự do: gia tốc và con quay hồi chuyển), thường thông qua giao thức I2C.

Cấu hình MPU6050:

Thiết lập các thanh ghi (registers) của MPU6050, bao gồm:

Chọn nguồn xung clock.:

Thiết lập dải đo (full-scale range) cho gia tốc kế và con quay hồi

Thiết lập bộ lọc kỹ thuật số (DLPF) nếu cần.

Khởi tạo MPU thành công:

Kiểm tra trạng thái sau khi cấu hình.

Nếu KHÔNG (no): Quá trình khởi tạo thất bại.

Delay 1000ms: Hệ thống chờ 1 giây.

Sau đó quay lại bước Kết nối MPU6050 để thử lại.

Nếu CÓ (yes): Quá trình khởi tạo thành công, chuyển sang vòng lặp đọc dữ liệu chính.

Đọc dữ liệu gia tốc:

Hệ thống đọc giá trị gia tốc hiện tại từ ba trục X, Y, Z của cảm biến MPU6050.

Kiểm tra đọc có bị lỗi:

Kiểm tra xem quá trình đọc dữ liệu từ cảm biến có xảy ra lỗi truyền thông (như lỗi I2C) hay không.

Nếu CÓ (yes): Lỗi đọc, bỏ qua các bước giải mã và quay lại Đọc dữ liệu gia tốc để thử lại.

Nếu KHÔNG (no): Dữ liệu hợp lệ, chuyển sang giải mã cử chỉ.

Kiểm tra nghiêng phải (trục Y > 0.4):

So sánh giá trị gia tốc trên trục Y với ngưỡng 0.4 (được coi là ngưỡng nghiêng phải).

Nếu CÓ (yes): Găng tay nghiêng sang phải.

Lệnh = 'R': Đặt lệnh điều khiển là 'R' (Right). Chuyển đến Kiểm tra điều kiện gửi.

Nếu KHÔNG (no): Tiếp tục kiểm tra điều kiện tiếp theo.

Kiểm tra nghiêng trái (trục Y < -0.4):

So sánh giá trị gia tốc trên trục Y với ngưỡng -0.4 (ngưỡng nghiêng trái).

Nếu CÓ (yes): Găng tay nghiêng sang trái.

Lệnh = 'L': Đặt lệnh điều khiển là 'L' (Left). Chuyển đến Kiểm tra điều kiện gửi.

Nếu KHÔNG (no): Tiếp tục kiểm tra điều kiện tiếp theo.

Kiểm tra tiến (trục X > 0.5):

So sánh giá trị gia tốc trên trục X với ngưỡng 0.5 (ngưỡng tiến).

Nếu CÓ (yes): Găng tay nghiêng về phía trước.

Lệnh = 'F': Đặt lệnh điều khiển là 'F' (Forward). Chuyển đến Kiểm tra điều kiện gửi.

Nếu KHÔNG (no): Tiếp tục kiểm tra điều kiện tiếp theo.

Kiểm tra lùi (trục X < -0.5):

So sánh giá trị gia tốc trên trục X với ngưỡng -0.5 (ngưỡng lùi).

Nếu CÓ (yes): Găng tay nghiêng về phía sau.

Lệnh = 'B': Đặt lệnh điều khiển là 'B' (Backward). Chuyển đến Kiểm tra điều kiện gửi.

Nếu KHÔNG (no): Nếu không thỏa mãn bất kỳ điều kiện nào (nghiêng X, Y), mặc định lệnh là 'S' (Stop).

Lệnh = 'S': Đặt lệnh điều khiển là 'S' (Stop). Chuyển đến Kiểm tra điều kiện gửi.

Kiểm tra điều kiện gửi:

Kiểm tra xem lệnh điều khiển hiện tại có khác với lệnh đã gửi lần trước hay không, hoặc có phải đã đủ thời gian để gửi lệnh mới hay chưa. Việc này giúp tránh gửi liên tục cùng một lệnh không cần thiết.

Nếu CÓ (yes): Lệnh mới cần được gửi đi.

Gửi notify qua BLE: Gửi lệnh điều khiển ('R', 'L', 'F', 'B', 'S') đến thiết bị Xe thông qua cơ chế BLE Notify.

Nếu KHÔNG (no): Lệnh hiện tại giống lệnh đã gửi lần trước, không cần gửi lại.

Cập nhật trạng thái:

Lưu lại lệnh vừa được gửi thành công (hoặc lệnh hiện tại nếu không gửi) để so sánh trong vòng lặp tiếp theo.

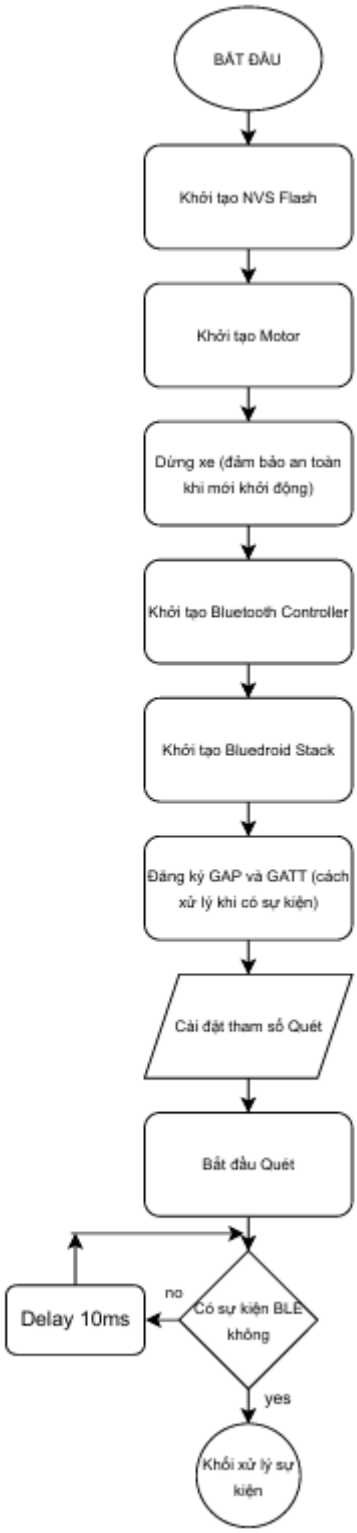
Delay 100ms:

Hệ thống tạm dừng 100 mili giây. Điều này thiết lập tần suất lấy mẫu và gửi lệnh (tối đa 10 lần/giây, tương đương 10Hz).

Sau khi kết thúc Delay, quay lại Đọc dữ liệu gia tốc để bắt đầu chu kỳ điều khiển mới.

### *4.3 Lưu đồ giải thuật xe*

**a) Lưu đồ giải thuật tổng quát**




Bắt đầu:

Điểm khởi đầu của chương trình trên thiết bị xe.

Khởi tạo NVS Flash:

Thực hiện khởi tạo bộ nhớ không bay hơi NVS (Non-Volatile Storage) Flash. Đây là bước cần thiết để lưu trữ dữ liệu vĩnh viễn, đặc biệt là các thông số cấu hình của Bluetooth/WiFi, trước khi khởi động giao tiếp không dây.

Khởi tạo Motor:

Thiết lập các chân điều khiển động cơ (motor control pins) trên ESP32, bao gồm các chân PWM (Pulse Width Modulation) và các chân điều khiển logic (IN1, IN2) để giao tiếp với Driver TB6612FNG.

Dừng xe (đảm bảo an toàn khi mới khởi động):

Để đảm bảo an toàn ngay khi bật nguồn, hệ thống thiết lập tất cả các động cơ ở trạng thái Dừng (Stop). Điều này ngăn xe tự ý di chuyển trước khi nhận được lệnh điều khiển hợp lệ qua BLE.

Khởi tạo Bluetooth Controller:

Khởi động tầng điều khiển vật lý (Controller layer) của Bluetooth trên ESP32, chuẩn bị cho việc giao tiếp không dây.

Khởi tạo Bluedroid Stack:

Khởi động ngăn xếp phần mềm Bluedroid Stack, cho phép ứng dụng sử dụng các hàm BLE ở cấp độ cao hơn để thực hiện các thao tác quét, kết nối và nhận/gửi dữ liệu.

Đăng ký GAP và GATT (cách xử lý khi có sự kiện)

Thiết bị xe cần đăng ký các hàm xử lý sự kiện cho các giao thức chính:

GAP (Generic Access Profile): Đăng ký các sự kiện liên quan đến quét (scanning), quảng bá (advertising) và kết nối.

GATT (Generic Attribute Profile): Đăng ký các sự kiện liên quan đến việc nhận dữ liệu (Notify) từ Server (Găng tay), đọc các Service và Characteristic.

Cài đặt tham số Quét:

Cấu hình các tham số cần thiết cho quá trình tìm kiếm thiết bị Găng tay, bao gồm:

Khoảng thời gian quét (Scan interval).

Cửa sổ quét (Scan window).

Thiết lập để chỉ quét các thiết bị BLE mong muốn (có thể lọc theo tên hoặc địa chỉ MAC).

Bắt đầu Quét:

Thiết bị xe bắt đầu quá trình tìm kiếm các thiết bị BLE lân cận (quá trình này được gọi là discovery) để tìm ra thiết bị Găng tay (BLE Server) và thiết lập kết nối.

Có sự kiện BLE không

Đây là vòng lặp chính (main loop) của thuật toán. Hệ thống liên tục kiểm tra xem có sự kiện Bluetooth Low Energy mới xảy ra hay không. Các sự kiện này có thể là:

Tìm thấy thiết bị Găng tay.

Kết nối thành công hoặc bị mất kết nối.

Nhận được lệnh điều khiển (Notify) từ Găng tay.

Nếu KHÔNG (no): Không có sự kiện nào.

Delay 10ms: Hệ thống chờ 10 mili giây để tránh lãng phí tài nguyên CPU và sau đó kiểm tra lại.

Nếu CÓ (yes): Một sự kiện BLE đã xảy ra.

Khối xử lý sự kiện:

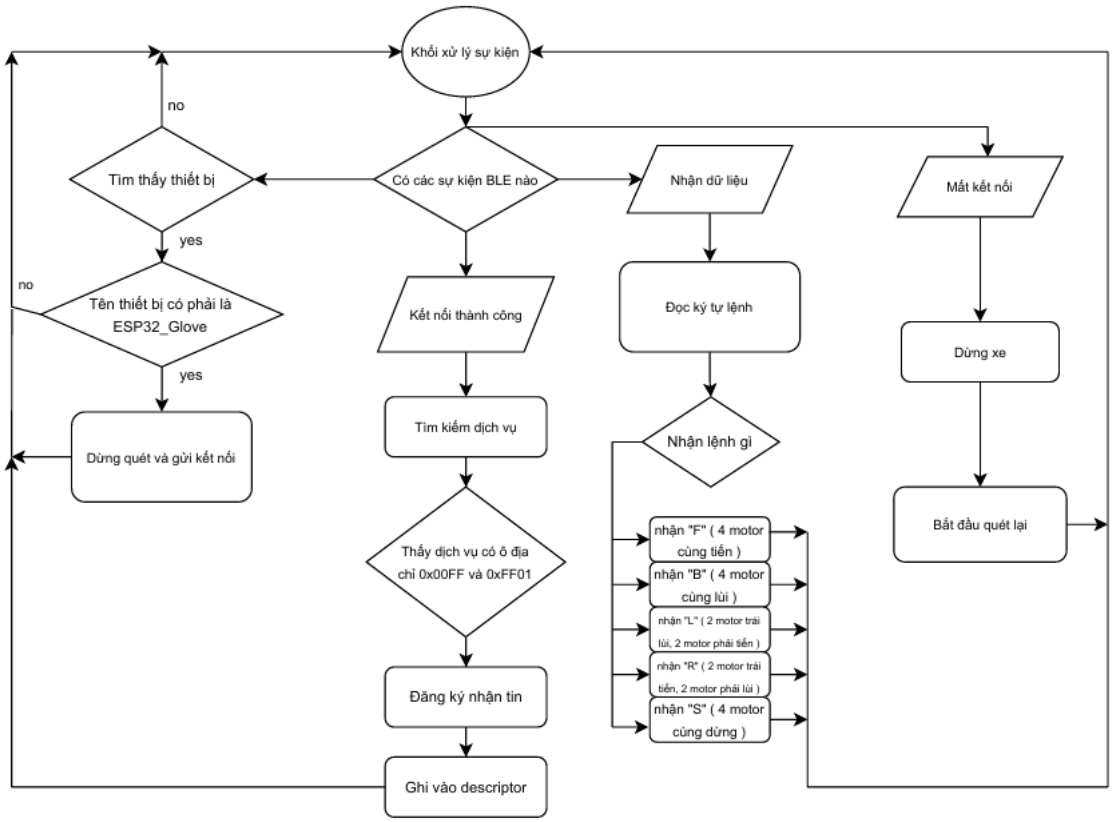
Thực hiện logic cần thiết để xử lý sự kiện BLE đã nhận được:

Nếu là sự kiện Tìm thấy thiết bị: So sánh với thông tin của Găng tay (tên, địa chỉ), nếu đúng thì tiến hành kết nối.

Nếu là sự kiện Nhận Notify/Write: Giải mã lệnh điều khiển (ví dụ: 'F', 'B', 'L', 'R', 'S') và gọi hàm điều khiển động cơ (Motor Control function) để thực hiện lệnh Tiến, Lùi, Rẽ hoặc Dừng.

Sau khi xử lý xong sự kiện, hệ thống quay trở lại Có sự kiện BLE không để tiếp tục kiểm tra và chờ đợi các sự kiện tiếp theo.

**b) Lưu đồ giải thuật chi tiết**




Phân loại Sự kiện BLE:

Khối Có các sự kiện BLE nào phân loại loại sự kiện vừa xảy ra:

Sự kiện Tìm thấy thiết bị: Xảy ra khi thiết bị xe đang trong quá trình quét và phát hiện ra một thiết bị BLE lân cận.

Sự kiện Kết nối thành công: Xảy ra sau khi xe đã tìm thấy găng tay và đã thiết lập kết nối không dây thành công.

Sự kiện Nhận dữ liệu: Xảy ra khi thiết bị Găng tay gửi thông báo (Notify) chứa lệnh điều khiển.

Sự kiện Mất kết nối: Xảy ra khi kết nối BLE với Găng tay bị ngắt đột ngột hoặc được ngắt theo quy trình.

Xử lý Sự kiện Tìm thấy thiết bị

Nếu sự kiện là Tìm thấy thiết bị:

Tên thiết bị có phải là ESP32_Glove: Hệ thống kiểm tra tên (hoặc địa chỉ) của thiết bị vừa tìm thấy.

Nếu CÓ (yes): Thiết bị đó chính là Găng tay.

Dừng quét và gửi kết nối: Dừng quá trình quét các thiết bị khác và bắt đầu gửi yêu cầu kết nối đến Găng tay.

Nếu KHÔNG (no): Thiết bị lạ, quay lại Xử lý sự kiện để chờ sự kiện tiếp theo (tiếp tục quét).

Xử lý Sự kiện Kết nối thành công

Nếu sự kiện là Kết nối thành công:

Tìm kiếm dịch vụ: Xe bắt đầu thăm dò các dịch vụ (Service) mà Găng tay (Server) cung cấp.

Thấy dịch vụ ở địa chỉ 0x00FF và 0xFF01: Sau khi tìm kiếm, hệ thống kiểm tra xem có tìm thấy Service đặc trưng cho dữ liệu điều khiển từ Găng tay hay không (ví dụ: với UUID là 0x00FF và Characteristic là 0xFF01).

Nếu CÓ (yes): Tìm thấy kênh dữ liệu hợp lệ.

Đăng ký nhận tin (Notify): Xe đăng ký nhận thông báo (Notify) từ Characteristic này. Đây là cơ chế để Găng tay chủ động gửi lệnh khi có thay đổi.

Ghi vào Descriptor: Ghi giá trị (thường là 0x0001) vào Client Characteristic Configuration Descriptor (CCCD) của Characteristic để kích hoạt tính năng Notify từ phía Server.

Nếu KHÔNG (no): Dịch vụ không hợp lệ, quay lại Xử lý sự kiện.

Xử lý Sự kiện Nhận dữ liệu (Lệnh điều khiển)

Nếu sự kiện là Nhận dữ liệu:

Đọc ký tự lệnh: Đọc giá trị ký tự lệnh (thường là 'F', 'B', 'L', 'R', 'S') được gửi trong thông báo Notify.

Nhận lệnh gì: Phân tích ký tự lệnh để điều khiển động cơ:

Nhận 'F': 4 motor cùng tiến.

Nhận 'B': 4 motor cùng lùi.

Nhận 'L': 2 motor phải lùi, 2 motor trái tiến (để xe rẽ trái nhanh hơn).

Nhận 'R': 2 motor trái lùi, 2 motor phải tiến (để xe rẽ phải nhanh hơn).

Nhận 'S': 4 motor cùng dừng.

Sau khi thực hiện lệnh, quay lại Xử lý sự kiện để chờ lệnh tiếp theo.

Xử lý Sự kiện Mất kết nối

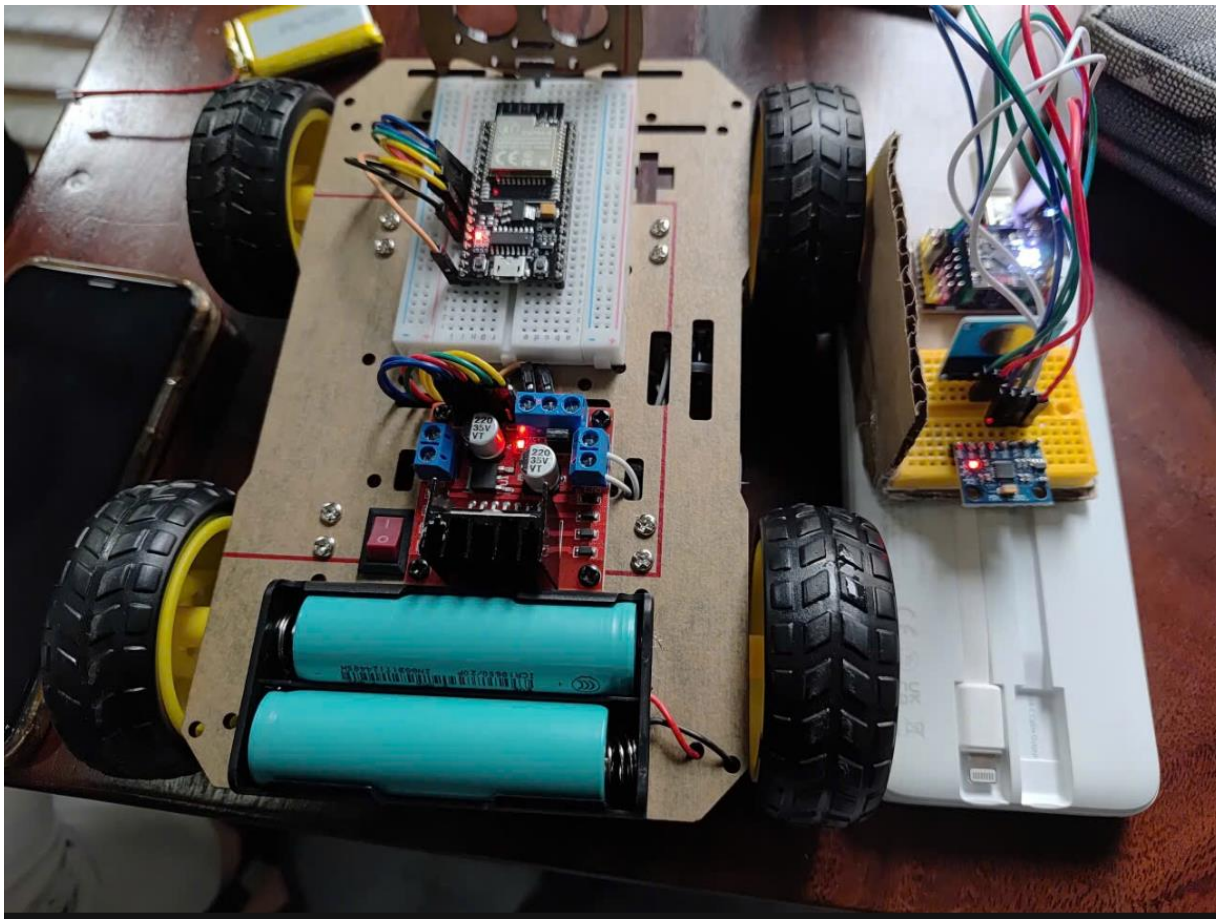
Nếu sự kiện là Mất kết nối:

Dừng xe: Để đảm bảo an toàn, hệ thống ngay lập tức đặt 4 động cơ về trạng thái Dừng (Stop).

Bắt đầu quét lại: Thiết bị xe chuyển trạng thái từ Client đã kết nối trở lại chế độ quét để tìm kiếm và kết nối lại với Găng tay.

Sau đó quay lại Xử lý sự kiện để chờ các sự kiện tiếp theo.

## Chương 5: Kết quả thực hiện



### *5.1 Tiến hành thử nghiệm:*

Hiệu chuẩn cảm biến:

Đặt găng tay trên mặt phẳng ngang, hiệu chuẩn MPU6050 để xác định giá trị gia tốc (X, Y, Z) ở trạng thái dừng (stop) và thiết lập ngưỡng nghiêng

Kiểm tra kết nối BLE: Khởi động cả hai module ESP32. Thiết bị Găng tay đóng vai trò là BLE Server và thiết bị Xe là BLE Client. Đảm bảo kết nối BLE được thiết lập và duy trì liên tục trong phạm vi 5-10 mét.

Thử nghiệm từng cử chỉ:

Người điều khiển thực hiện từng cử chỉ: Nghiêng phải, Nghiêng trái, Nghiêng tiến, Nghiêng lùi, Giữ thẳng (Dừng).

Theo dõi thiết bị Xe có nhận và thực hiện đúng lệnh R, L, F, B, S tương ứng hay không.

Đo độ trễ (Latency): Dùng đồng hồ bấm giờ đo thời gian từ khi người điều khiển thực hiện cử chỉ dứt khoát đến khi xe bắt đầu chuyển động theo lệnh.

Thử nghiệm liên tục: Thực hiện điều khiển xe di chuyển liên tục trong một không gian nhỏ, thử nghiệm khả năng chuyển lệnh và thay đổi hướng, tốc độ.

### *5.2 Kết quả thí nghiệm*

**a) Kết quả thử nghiệm độ chính xác nhận lệnh**

Kết quả thử nghiệm độ chính xác nhận lệnh được trình bày như sau. Đối với lệnh Tiến (F), cử chỉ thực hiện là Nghiêng tiến (Trục X lớn hơn 0.5). Trong 100 lần thử nghiệm, số lần nhận lệnh đúng là 96, đạt tỷ lệ chính xác 96%. Đối với lệnh Lùi (B), cử chỉ thực hiện là Nghiêng lùi (Trục X nhỏ hơn -0.5). Trong 100 lần thử nghiệm, số lần nhận lệnh đúng là 95, đạt tỷ lệ chính xác 95%. Đối với lệnh Rẽ Trái (L), cử chỉ thực hiện là Nghiêng trái (Trục Y nhỏ hơn -0.4). Trong 100 lần thử nghiệm, số lần nhận lệnh đúng là 97, đạt tỷ lệ chính xác 97%. Đối với lệnh Rẽ Phải (R), cử chỉ thực hiện là Nghiêng phải (Trục Y lớn hơn 0.4). Trong 100 lần thử nghiệm, số lần nhận lệnh đúng là 94, đạt tỷ lệ chính xác 94%. Cuối cùng, đối với lệnh Dừng (S), cử chỉ thực hiện là Giữ thẳng. Trong 100 lần thử nghiệm, số lần nhận lệnh đúng là 98, đạt tỷ lệ chính xác 98%.

**b) Số liệu độ trễ phản hồi (trung bình của 30 lần đo)**

Số liệu độ trễ phản hồi được đo trung bình qua 30 lần đo cho thấy. Đối với các lệnh Tiến và Lùi, độ trễ trung bình là 250 mili giây (ms), độ trễ tối đa là 320 mili giây (ms), và yêu cầu hiệu năng đặt ra là nhỏ hơn hoặc bằng 500 mili giây (ms). Đối với các lệnh Rẽ Trái và Rẽ Phải, độ trễ trung bình là 280 mili giây (ms), độ trễ tối đa là 350 mili giây (ms), và yêu cầu hiệu năng đặt ra là nhỏ hơn hoặc bằng 500 mili giây (ms).

### *5.3 Phân tích kết quả thử nghiệm*

Tỷ lệ Chính xác: Tỷ lệ chính xác nhận lệnh trung bình đạt khoảng 96%. Điều này cho thấy hệ thống giải mã cử chỉ hoạt động hiệu quả với các ngưỡng đã thiết lập. Tỷ lệ lỗi thấp (khoảng 2% - 6%) thường xảy ra do nhiễu khi đọc cảm biến hoặc khi người điều khiển thực hiện cử chỉ không dứt khoát, nằm sát ngưỡng so sánh.

Độ trễ: Độ trễ trung bình của toàn hệ thống nằm trong khoảng 250ms - 280ms. Điều này bao gồm: thời gian đọc MPU6050, thời gian giải mã cử chỉ, thời gian truyền BLE Notify, và thời gian ESP32 (Car) xử lý lệnh và kích hoạt driver động cơ.

Phân tích các số liệu để biết kết quả đã thực hiện là phù hợp, đạt yêu cầu:

Đáp ứng Yêu cầu Hiệu năng: Độ trễ trung bình 250-280ms thấp hơn đáng kể so với yêu cầu đặt ra. Điều này đảm bảo khả năng điều khiển xe một cách thời gian thực và mượt mà.

Đảm bảo Chức năng: Với tỷ lệ chính xác trên 94% cho tất cả các lệnh, hệ thống đã đáp ứng được yêu cầu chức năng cơ bản là "Tạo một hệ thống xe robot 4 bánh có thể điều khiển từ xa bằng cử chỉ tay".

# Chương 6: Kết luận và hướng phát triển

## 6.1 Kết luận

Đề tài đã hoàn thành mục tiêu đặt ra là thiết kế và chế tạo thành công một hệ thống xe robot 4 bánh điều khiển từ xa bằng cử chỉ tay (găng tay cảm biến), sử dụng hai module ESP32 để giao tiếp qua Bluetooth Low Energy (BLE).

Ưu điểm:

Hệ thống hoạt động với độ trễ thấp (khoảng 250-280ms) và tỷ lệ chính xác nhận lệnh cao (trung bình 94%) nhờ vào việc sử dụng cảm biến IMU MPU6050 để giải mã cử chỉ.

Sử dụng các linh kiện phổ biến và chi phí hợp lý, với tổng chi phí dự kiến khoảng 750.000 VNĐ.

Thể hiện được tính ứng dụng trong việc điều khiển không dây bằng cử chỉ và phục vụ cho mục đích học tập, nghiên cứu.

Khuyết điểm:

Phạm vi điều khiển bị giới hạn bởi sóng Bluetooth/WiFi (dự kiến 5-10 mét).

Chỉ sử dụng MPU6050 để nhận diện hướng nghiêng, chưa tích hợp cảm biến uốn (flex sensor) để tăng cường độ phức tạp và chi tiết của cử chỉ.

## 6.2 Hướng phát triển

Hướng phát triển chính của đề tài là nâng cấp hệ thống điều khiển lên tầm nhìn thứ nhất (First-Person View - FPV) để tăng cường trải nghiệm và phạm vi ứng dụng.

Tích hợp Camera FPV: Gắn một module camera (ví dụ: ESP32-CAM) phía trước xe để thu thập hình ảnh hoặc video trực tiếp.

Truyền dữ liệu video: Sử dụng giao thức WiFi (thay vì BLE) để truyền dữ liệu video về một thiết bị nhận (như điện thoại thông minh, máy tính bảng hoặc một màn hình FPV chuyên dụng) ở phía người lái.

Điều khiển từ xa mở rộng: Người lái sẽ điều khiển xe thông qua găng tay bằng cách quan sát hình ảnh trực tiếp từ camera mà không cần nhìn thấy chiếc xe. Điều này cho phép điều khiển xe ở những khu vực khuất tầm nhìn hoặc xa hơn, mở rộng phạm vi ứng dụng của hệ thống robot.

## Tài liệu tham khảo

1. Đỗ Văn Đỉnh, Thiết kế hệ thống nhúng-Lý thuyết và ứng dụng, Nhà xuất bản Khoa học và Kỹ Thuật, 2005.

## Phụ lục

Code phát: https://github.com/m1nh1512/ESP32_Gesture_Car_control_part

Code thu: https://github.com/m1nh1512/ESP32_Gesture_Car_part